

Data Structures ++

Workshop – Term 2, Week 5

Matthew Gu, Holly Field, Austin Song

Outline

1 Housekeeping

2 Heaps

3 Difference arrays

4 Sets and hashmaps

5 Segment trees

- Revision of basic segtrees
- Sparse segment trees
- Range updates, point queries
- Non-standard monoids

6 Appendix and resources

Attendance!

Please scan the attendance form so you can get free food



VJudge For Workshop

If you're confident with the content for today, feel free to get started on problems! Link:

<https://vjudge.net/contest/826177>

Outline

1 Housekeeping

2 Heaps

3 Difference arrays

4 Sets and hashmaps

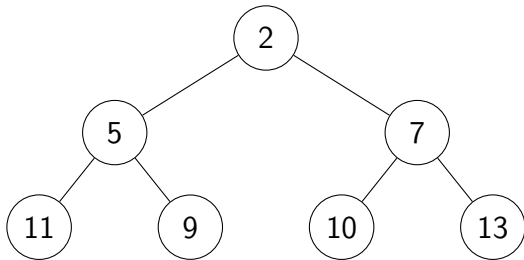
5 Segment trees

- Revision of basic segtrees
- Sparse segment trees
- Range updates, point queries
- Non-standard monoids

6 Appendix and resources

Under the hood

A **heap** is really a **binary tree**, with the rule that every node is at least as good as its children.



This is a **min-heap**: every node is smaller than or equal to its children.

Priority Queues in C++

Usual max-heap

```
priority_queue<int> pq;
```

Min-heap

```
priority_queue<int, vector<int>, greater<int>> pq;
```

Custom comparator

```
struct Item {  
    int priority, id;  
  
    bool operator<(const Item& other) const {  
        return priority > other.priority;  
    }  
};
```

```
priority_queue<Item> pq;
```

Operations

Operation	Time Complexity
<code>pq.push(x)</code>	$O(\log n)$
<code>pq.pop()</code>	$O(\log n)$
<code>pq.top()</code>	$O(1)$
<code>pq.empty()</code>	$O(1)$
<code>pq.size()</code>	$O(1)$

Problem

Rent a Labubu

There is one extremely questionable Labubu rental shop.

There are N customers. Customer i wants to rent a Labubu from day a_i to day b_i , inclusive.

Two customers can use the same Labubu if the first customer returns it before the second customer starts renting it.

Find the minimum number of Labubus the shop needs.

Constraints:

- $1 \leq N \leq 2 \cdot 10^5$
- $1 \leq a_i \leq b_i \leq 10^5$

Sample

Sample Case

Sample Input:

```
5
1 3
2 5
4 6
6 6
7 8
```

Sample Output:

```
3
```

Solution

Let's try to track all **active customers**, since the maximum number of active customers is exactly the number of Labubus needed.

- Process customers in increasing order of start day a_i .
- We want to quickly remove customers who already returned their Labubu.
- So, store active customers in a **min-heap by return day**.
- Before customer (a_i, b_i) starts, pop while:

$$\text{pq.top}() < a_i$$

- Then push b_i into the heap.
- Take the maximum heap size over the whole sweep.

Time Complexity: $O(N \log N)$.

Outline

1 Housekeeping

2 Heaps

3 Difference arrays

4 Sets and hashmaps

5 Segment trees

- Revision of basic segtrees
- Sparse segment trees
- Range updates, point queries
- Non-standard monoids

6 Appendix and resources

Problem

Terrible Tan Lines

Your body has N patches of human skin in a line, numbered 1 to N .

You start off completely pale. Over the day, the sun shines on ranges of skin. Each event gives three integers l_i, r_i, x_i , meaning every patch from l_i to r_i gets x_i darker.

After all Q events, output the final tan level of every patch.

Constraints:

- $1 \leq N, Q \leq 2 \cdot 10^5$
- $1 \leq l_i \leq r_i \leq N$
- $1 \leq x_i \leq 10^9$

Sample

Sample Case

Sample Input:

```
5 3
1 3 2
2 5 4
4 4 1
```

Sample Output:

```
2 6 6 5 4
```

Can we do better than $O(NQ)$?

Derivatives of Arrays?

Let the tannedness array be a .

Consider the "derivative" of this array a' , where

$$a'_i = a_i - a_{i-1}.$$

What effect does doing an update do to these two arrays?

a	0	+x	+x	+x	+x	0
a'	0	+x	0	0	0	-x

We can recover a by "integrating" a' :

$$a_i = a_{i-1} + a'_i$$

Solution

Store updates on the derivative array a' .

- To add x to the range $[l, r]$:

$$a'_l += x$$

$$a'_{r+1} -= x$$

- After processing all Q updates, recover a by prefix sums:

$$a_i = a_{i-1} + a'_i$$

- Then output all values of a .

Time Complexity: $O(N + Q)$.

Labubu Rental (Redux)

We can also solve the problem using difference arrays!

Each rental $[a_i, b_i]$ adds one active customer on every day in that range.

- So we apply a range add of $+1$:

$$\text{diff}_{a_i} += 1$$

$$\text{diff}_{b_i+1} -= 1$$

- Prefix sum over `diff` to get the number of active customers each day.
- The maximum active customers on any day is the number of Labubus needed.

Time Complexity: $O(N + \max a_i)$.

Outline

1 Housekeeping

2 Heaps

3 Difference arrays

4 Sets and hashmaps

5 Segment trees

- Revision of basic segtrees
- Sparse segment trees
- Range updates, point queries
- Non-standard monoids

6 Appendix and resources

C++ Data Structures

- `<set>` / `<multiset>`: ordered set of unique / duplicate values with $O(\log(n))$ operations:
 - `.insert()` to insert an element
 - `.erase()` to remove an element
 - `.count()` or `.contains()` to check existence of an element
- `<map>`: ordered set of key value pairs with $O(\log(n))$ operations:
 - `.insert({key, val})` to insert a key-value pair
 - `.erase({key, val})` to remove a key-value pair
 - `map[key]` to access the value for a key

Bound Methods

Sets and maps maintain items **in order**, hence have internal `upper_bound()` and `lower_bound()` methods where:

- `.lower_bound(val)` returns an iterator to the first element $\geq val$. You can shift this iterator left one using `iter-` to then access the first element smaller than `val`.
- `.upper_bound(val)` returns an iterator to the first element $> val$

These methods come especially in handy for greedy algorithms, where the "best choice" is determined by some value closest to a target.

In general:

- Sets and maps are useful when you need to query something in $\log(n)$ time.
- If the problem requires an $O(n \log n)$ solution, this might hint towards a single sweep, with a set or map operation for each iteration!

Set Problem

<https://cses.fi/problemset/task/1084>

Apartments

There are n applicants and m free apartments. Your task is to distribute the apartments so that as many applicants as possible will get an apartment.

Each applicant has a desired apartment size, and they will accept any apartment whose size is close enough to the desired size (with a maximum allowed difference of k)

Print one integer: the number of applicants who will get an apartment.

Constraints:

■ $1 \leq n, m \leq 2 \cdot 10^5, 0 \leq k \leq 10^9, 1 \leq a_i, b_i \leq 10^9$

Samples

Sample Cases

Sample Input:

4 3 5

60 45 80 60

30 60 75

Sample Output:

2

Solution

To **maximise** the number of applicants that get an apartment, we can greedily allocate the **smallest apartment that fits within each applicant's size preferences**.

To do this, we can use a **set**!

- 1 Add all apartment sizes to a set
- 2 Use `lower_bound()` to get the first apartment $\geq \text{preference} - k$
- 3 Check that this apartment fits within $\text{preference} + k$
- 4 If it is a match, increment the number of matched apartments and remove the apartment once it has been assigned.

However, we must be careful as an applicant with a larger apartment preference may "steal" a small apartment that could have been allocated otherwise. So, we should **sort** applicants by increasing order of desired size to ensure we process them in increasing order, so smaller preferences are matched first.

Map Problem

<https://cses.fi/problemset/task/3222>

Sliding Window Distinct Values

You are given an array of n integers. Your task is to calculate the number of distinct values in each window of k elements, from left to right.

Print: $n - k + 1$ values: the numbers of distinct values.

Constraints:

- $1 \leq k \leq n \leq 2 \cdot 10^5$

Samples

Sample Cases

Sample Input:

```
8 3
1 2 3 2 5 2 2 2
```

Sample Output:

```
3 2 3 2 2 1
```

Solution

Surprise! This problem needs a sliding window XD, but how should we store the state to quickly calculate the number of distinct values?

Well, for each window, we need to know:

- how many **unique** values there are, and
- the **frequency** of each value.

This aligns really well with a **map**!

- Keys are **unique**, so assigning input values as a keys allows us to get the number of distinct values in a window using `map.size()`.
- We can also track and update the frequencies of an input value by assigning them as the map values!

Solution

Bringing these ideas together:

- 1 Initialise the map with the first k values.

Then, as we slide the window across:

- 2 Increment the count of the value entering the window.
- 3 Decrement the count of the value leaving the window.
 - If the count is 0, remove it from the map with `map.erase()`.

Outline

1 Housekeeping

2 Heaps

3 Difference arrays

4 Sets and hashmaps

5 Segment trees

- Revision of basic segtrees
- Sparse segment trees
- Range updates, point queries
- Non-standard monoids

6 Appendix and resources

Range Queries

Dynamic Array Sums

You're given an array A of length N .

Furthermore, you're provided Q queries – the i 'th one either asks for the sum of elements between indices l_i and r_i inclusive, or asks to change the p_i 'th value to v_i .

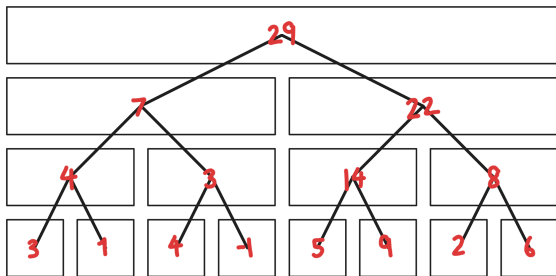
There are two common techniques to solve this kind of problem:

- We can simply **iterate** for every query.
 - Update: $O(1)$.
 - Query: $O(N)$.
- We can also utilise **prefix sums**!
 - Query: $O(1)$.
 - Update: $O(N)$.

Is there a way we can achieve better than $O(QN)$ complexity?

Segment (Range) Trees

Prefix sums gain their query efficiency from precomputing range sums – let's take inspiration!



Observation I: Any range can be decomposed into at most $\log N$ disjoint ranges.

Observation II: Any update causes changes to at most $\log N$ nodes.

We can therefore traverse the tree to appropriately update and gather nodes in $O(\log N)$!

Applications

In general, this data structure is called a **segment tree** (or colloquially a **segtree**).

Segtrees are super versatile! As long as you have any quickly-computable associative operation – i.e.

$$a \circ b \circ c \circ d = (a \circ b) \circ (c \circ d),$$

you can simply store the answer for a bunch of ranges, and update accordingly!

Examples of things you can do:

- Query $\max([l \dots r])$, set index i to value v .
- Query $\gcd([l \dots r])$, add v to the value at index i .
- Query the frequency of an element x in a range $[l \dots r]$, set index i to value v (how?)
- and more... (note the prolepsis)

Tips for Programming Segtrees!

In general, the scaffold for querying and updating looks mostly the same for varying segtrees, so it's good to practice and become fluent with writing the update/query traversals.

- Think what you want your underlying 'array' to be.
- Know what property you'd like to store for ranges.
- Our tree can be built implicitly by creating an array of nodes:
 - A node with index i has children at $2i$ and $2i + 1$.
 - This means a maximum node-array size of $4N$ is needed, where N is the size of the implicit underlying array.
- You can build in $O(N)$ if you write another function, but in practice we can just update N times for $O(N \log N)$ build (who cares)
- Might be more intuitive to use inclusive-inclusive ranges governing $[l, r]$ instead of $[l, r)$.

Time to demonstrate!

Massive Number Line

Bug Catching

You monitor bugs which appear on a number line with positions from 1 to L . Support Q operations:

- 1 Discover a new bug at position x_i .
- 2 Delete the bug at position x_i .
- 3 Return the number of bugs between positions l_i and r_i inclusive.

Constraints:

- Subtask 1 (50%): $1 \leq Q \leq 2 \times 10^5$, $1 \leq L \leq 2 \times 10^5$.
- Subtask 2 (50%): $1 \leq Q \leq 2 \times 10^5$, $1 \leq L \leq 10^9$.

Bug Catching – Subtask 1 Solution

Let's 'build' an array over our number line, which stores the number of bugs at a certain position.

Then, our two updates reduce to:

- Add a value $v \in \{-1, 1\}$ to an index/position x .

And our query becomes:

- Find the range sum of subarray $[l \dots r]$.

Hopefully, everyone recognises this can be done with a **range-sum point-add segtree!**

Question: What's our final time complexity?

Answer: It's $O(Q \log L)$, which is fast enough for both Subtask 1 and Subtask 2.

So why doesn't our solution work for the full problem?

Bug Catching – Oops

Unfortunately, the issue we run into isn't time – $O(Q \log L)$ is fine for large L – it's **memory**!

- We need to allocate at least $4L$ items in our segtree.
- For 32-bit integer types, this means ~ 16 GB of memory!

How can we deal with this? (Not a rhetorical question, does anyone know how to deal with this?)

Sparse Segment Trees

A key realisation is that we only care about at most Q distinct positions over all queries!

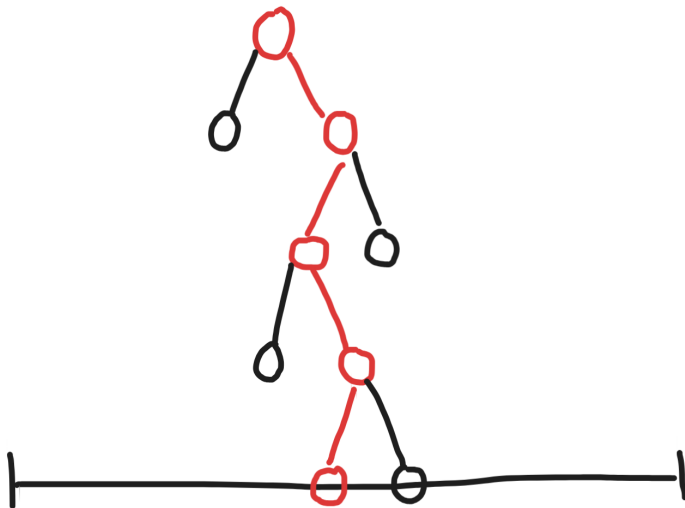
- It doesn't matter that our 'array' goes to $L = 10^9$, if we only perform $Q = 3$ queries adding bugs – everything else is 0 and nothing interesting happens!

So, we only need Q leaf nodes in the worst case on our segtree. Since our depth is still $O(\log L)$, our (theoretical) memory usage is $O(Q \log L)$.

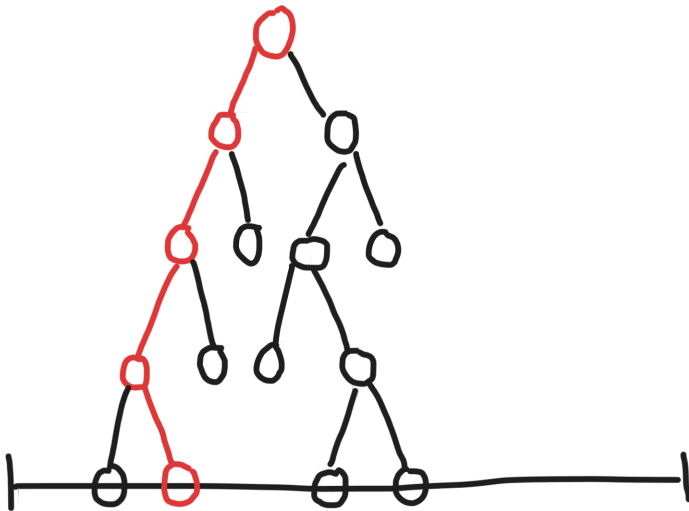
How do we actually achieve this memory?

- Firstly, let's ditch the $i \rightarrow \{2i, 2i + 1\}$ inefficient mapping.
- Instead, we'll store the indices of children **at the parent**.
- We only 'make' the children of a node when we visit it.
- This means each query makes $O(\log L)$ nodes, so we achieve $O(Q \log L)$ memory total!

Sparse Segtrees – Terrible Drawing



Sparse Segtrees – Terrible Drawing



Range Updates?

Planting Crops

There's a field with N plots in a line. There are D days. On each day, either:

- All the plots from l_i to r_i inclusive grow v_i crops.
- You harvest the x_i 'th plot, setting it to 0.

For each harvest, output the number of crops you obtain.

Constraints:

- $1 \leq N, D \leq 2 \times 10^5$.

Planting Crops – Solution

We need to do the following:

- Update a range $[l \dots r]$ by incrementing by v in $O(\log N)$.
- Query the value of index x in $O(\log N)$.

(Can we see how these operations are enough to solve the problem?)

To do this, there's some dynamic difference-array tech you can do... but here's another trick!

- Let's have each node of the segtree store the increment applied to (exactly) *that* range.
 - To update, we find ranges that cover $[l \dots r]$ and $+v$ to their values.
 - To query, we accumulate the values of nodes which lie on the path to x .
- So we can just swap our update and query traversals :)
 - That is, for `qry(x)`, we find the path of nodes to the leaf at x .
 - And `upd(l, r, v)` will find the set of nodes that cover $[l \dots r]$.

Boom, range-update/point-query segtree <0>_<0>

Monoids

I don't know what this word means :)

Associative Thingos

Segtrees can be built over any objects where the **merge** operation is associative!

- e.g. Since we can get the sum of a big range $[l \dots r]$ as the sum of $[l \dots m]$ and $[m + 1 \dots r]$, we can build a segtree to query for the sum of a range!
- Similarly, min/max/gcd can all be merged in $O(1)$ or $O(\log N)$.
- You can even store `std::set<T>`s at each node!

So in general, segtrees just need a valid (and fast) merge function, which takes two children's objects and turns them into a parent object:

$$\text{merge}(f([l \dots m]), f([m + 1 \dots r])) \rightarrow f([l \dots r]).$$

But there are **many** things that are merge-able...

Simple Monoids

MNASS

You have an array A of length N . Support the following Q operations:

- Query the **maximum non-adjacent subsequence sum** (alt. MNASS) of elements between indices $[l_i \dots r_i]$.
- Update the element at x_i to value v_i .

For instance, the MNASS of $[4, 8, 5, -3, -1, 2]$ is $[4, 8, \mathbf{5}, -3, -1, \mathbf{2}] \rightarrow 11$.

Constraints:

- $1 \leq N, Q \leq 2 \times 10^5$.

MNASS – Solution

If we knew the MNASS of a range $[l \dots m]$ and the MNASS of a range $[m + 1 \dots r]$, is that sufficient to know $[l \dots r]$?

- Nope! If the MNASS of the right range used A_{m+1} and the left range used A_m , then the combined range wouldn't be valid.

So we need to store more information to help us merge...

What if we just bashed the cases? For each range, let's store:

- v_l : The MNASS including l but not r .
- v_r : The MNASS including r but not l .
- v_{lr} : The MNASS including both.
- v_0 : The MNASS excluding both.

Then, we can definitely merge! e.g.

$$[L, R].v_l = \max([L, M].v_l + [M + 1, R].v_l, [L, M].v_{lr} + [M + 1, R].v_0, [L, M].v_l + [M + 1, R].v_0).$$

A Harder Problem

Financial Trouble (Modified)

There are N events. The i 'th one changes your money by a_i . If you have strictly less than 0 money after an event, you incur a “borrow”, where your money gets set to 0.

Answer Q queries – how much final money do you have after running through the events $[l_i \dots r_i]$ with starting money m_i ?

Constraints:

- $1 \leq N, Q \leq 2 \times 10^5, -10^9 \leq a_i \leq 10^9$.

Financial Trouble – A Range Function

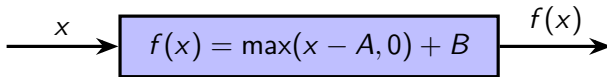
This is a hard problem!

- In a range, a lot of things happen.
- But eventually, you'll hit your **last borrow**, and then simply gain money after that.
- In other words, every range of events can be thought of as a function:

$$f_{A,B}(x) = \max(x - A, 0) + B,$$

for some constants A and B .

- If it helps, this simply means each range reduces to a $-A$ event, followed by a $+B$ event.



- This model still holds even after composition!
 - i.e. It'll always look like some $-A$ followed by a $+B$.

Financial Trouble – Solution

The output of the left range gets fed as the new starting money into the right range!

- We're just doing function composition :)
- Call the left function $f_{A_L, B_L}(x)$ and the right one $f_{A_R, B_R}(x)$.

The maths is a bit finicky, but since $f_{A_T, B_T}(x) = f_{A_R, B_R}(f_{A_L, B_L}(x))$, it follows that

$$\begin{aligned}A_T &= A_L + \max(A_R - B_L, 0), \\ B_T &= B_R + \max(B_L - A_R, 0).\end{aligned}$$

(Hint: Consider cases when the left range adds enough to overwrite the right range's deficit – i.e. $B_L \geq A_R$ and $B_L < A_R$.)

Either way, we now have a way of getting $f_{A, B}(x)$ for **any** range via merging on a segtree!

- So we can find the $\log N$ ranges for each query range, and compose them.
- Then, the answer is just $f_{A, B}(m_i)$.

Outline

1 Housekeeping

2 Heaps

3 Difference arrays

4 Sets and hashmaps

5 Segment trees

- Revision of basic segtrees
- Sparse segment trees
- Range updates, point queries
- Non-standard monoids

6 Appendix and resources

Variety Problems

There are a couple variety problems on the VJudge (starting from **Inversion Counting**).
Some of them are Codeforces problems (**Pillars** and **QED's Favourite Permutation**) – go to Codeforces to submit.

Feedback!

Please fill out feedback form so we know how to run future workshops :D



(a) Feedback form



(b) Attendance form

Photo time :)

